Ampersand, a story-development IDE for writers
By Team Ampersand
GitHub Repository URL: https://github.com/willburnsucla/Ampersand.git

William Burns	**Gabriel Sanchez**	**Hana Chloe Yoon**	**Ashley Wu**
UID: 205966518	*UID: 905970079*	*UID: 706188827*	*UID: 206130730*
Thomas McConnell	**Emily Zhang**	**Lam Luong**	
UID: 306063216	*UID: 706227466*	*UID: 906063609*	

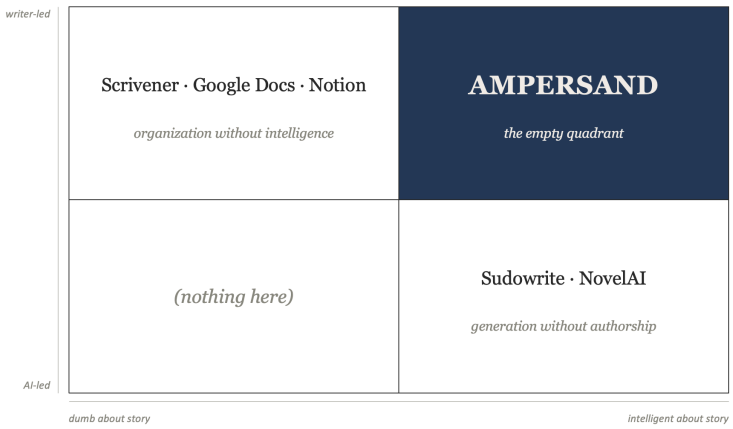
1. Context & Scope

The Problem

For most writers, and for those of us who write on our team, the hardest part of writing isn't prose. It's holding the whole shape of a story in their head while they develop it: characters and arcs, beats in order, thematic threads, world rules, branching what-ifs, open questions still hanging. The actual writing is often the easy part.

The tools writers use today don't help. Prose-production tools (Scrivener, Google Docs, Final Draft) don't model story structure; they're glorified typewriters. Generative AI tools (Sudowrite, NovelAI) try to write prose for the user, which most serious writers reject as a violation of authorship. Wiki-style organizers (World Anvil, Notion) let writers manually catalog story elements, but they don't understand the story or connect facts to the scenes that established them.

There's an empty quadrant: a tool that's intelligent about story structure but keeps the writer in charge of every creative decision. Ampersand is that quadrant. See Figure 1 for a visual of the landscape.



The Vision

Ampersand is a story-development environment, think an IDE, but for ideas. It's built on one idea: AI for creative work should help with what humans are bad at, not replace what they're good at. Humans are bad at holding many interconnected facts in their head and noticing when a new idea contradicts an old one. Humans are good at creativity, taste, and emotional resonance. Ampersand divides the work that way: the AI organizes, retrieves, surfaces, and checks; the writer creates.

A writer using Ampersand talks to a story-aware AI about their story. As they riff, the system extracts structured information from the conversation and builds a knowledge graph: characters, beats, world elements, themes, and the relationships between them. The graph renders through surfaces such as character pages, arc and tension visualizations, branch comparison views, a queryable conversational interface, with every fact traceable to the conversation that established it. The output of an extended session isn't prose; it's a structural skeleton the writer exports as Markdown and opens in the writing tool of their choice.

Domain Background & Terminology

A few concepts from narrative theory show up throughout this document:

- **Beat:** The smallest unit of dramatic change in a story is the moment a stakes-bearing decision, revelation, or shift occurs. A scene contains one or more beats; "she decides to leave him" is a beat, "they have dinner" usually isn't. Beats are the unit *Save the Cat*, *the Hero's Journey*, and *Story Grid* all converge on (these are story structuring frameworks), which is why they're the primary node type in our story graph: everything else (arcs, tension curves, thematic threads) is a function over the beat sequence.
- **Arc:** A character's trajectory across the beat sequence, usually described as a sequence of internal-state changes. Going from skeptic to believer, coward to committed, naive to disillusioned. An arc isn't stored directly in the graph; it's a derived view computed from the beats a character participates in and the state-changes attached to those beats.
- **Branch:** A live alternative possibility the writer is considering, such as, "what if she forgives him" versus "what if she doesn't." A branch isn't a version-control commit like Git; it's a parallel sketch of how the story could go from a given decision point. Writers can develop branches in parallel, compare them, commit to one, and keep the rest as a graveyard of explored ideas the system preserves but no longer propagates downstream.
- **Provenance:** Every fact in the story graph is tagged with the conversation turn that introduced it. This lets the writer see where a piece of their story came from, such as when they decided a character's profession, when a thematic motif first surfaced, when a beat was committed versus when it was still speculation. Provenance is what makes the graph trustworthy as a memory: the writer never has to wonder whether the system invented something.
- **Curatorial AI:** Ampersand's positioning, and the deliberate opposite of generative AI tooling. A generative AI produces creative content on the user's behalf, like drafting prose, inventing plot twists, or suggesting characters. A curatorial AI never does this. It only organizes, retrieves, surfaces, and checks what the user has already decided. The distinction matters because the two categories serve fundamentally different users: generative tools serve writers who want help producing, while curatorial tools serve writers who want help thinking. Ampersand is firmly in the second category.

2. Goals & Non-Goals

User Stories

US-1: Riff with the AI. As a writer with a half-formed idea, I want to talk to a story-aware AI about my story so that the system captures my decisions as structured graph nodes I can review later.

- Given an empty conversation, when the writer types a message describing a character, beat, theme, or world element, then the system responds in natural language and creates corresponding graph nodes within 5 seconds. Speculative statements ("what if...") are tagged proposed rather than committed.
- Given an existing graph, when the writer makes a statement that contradicts an earlier decision, then the system updates affected nodes and surfaces the change in its response.
- Dependencies: None. Foundational story.

US-2: Browse a character page. As a writer developing a character, I want to view a dedicated page showing that character's traits, arc, and beats they appear in.

- Given at least one character node, when the writer opens the Characters tab, then the system displays a list of all characters with names and one-line descriptions; clicking a character opens a page with traits, an arc visualization, and the beats they appear in.
- When the writer hovers any fact on the page, then the system shows a citation to the conversation turn that established it.

- Dependencies: Requires US-1.

US-3: Explore a branching what-if. As a writer exploring alternatives, I want to create a branch from any decision point and develop both possibilities in parallel.

- Given a beat with a defined outcome, when the writer initiates a branch, then the system creates a new exploration branch and routes subsequent conversation into it. Branch comparison displays divergent nodes side-by-side with shared upstream nodes hidden.
- When the writer commits to one branch, then chosen nodes are marked committed and the unchosen branch becomes a graveyard branch that no longer propagates.
- Dependencies: Requires US-1.

US-4: Export the story as a Markdown document. As a writer ready to start writing prose, I want to export my developed story as a stylized Markdown document.

- Given a populated graph, when the writer triggers export, then the system generates a Markdown file with sections for characters, beats (in order), themes, world elements, and open questions, with every fact traceable to its source via inline citation markers.
- Given multiple branches, the export defaults to the active branch with an option to include alternatives as appendices.
- Dependencies: Requires US-1. Optional integration with US-3.

US-5: See what the story is missing. As a writer planning my next session, I want the system to surface gaps in my story so I know where to focus.

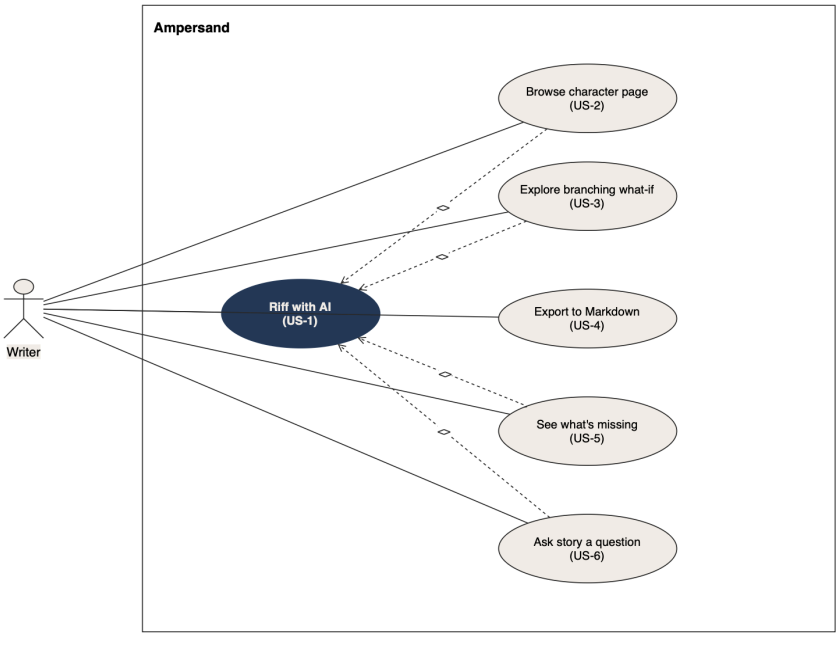
- Given gaps in the graph (open questions, undefined relationships, missing arc points), when the writer opens the Open Questions tab, then the system displays a categorized list. Structural gaps are also flagged in-context on the relevant page.
- When the writer clicks a gap entry, then the system opens the conversation interface with a leading question scoped to that gap.
- Dependencies: Requires US-1 and US-2.

US-6: Ask the story a question. As a writer with significant material accumulated, I want to ask the system natural-language questions about my story.

- Given a populated graph, when the writer asks a factual or analytical question, then the system returns an answer grounded in the graph with citations to the relevant nodes.
- When the question's answer isn't in the graph, then the system says so explicitly rather than hallucinating.
- Dependencies: Requires US-1.

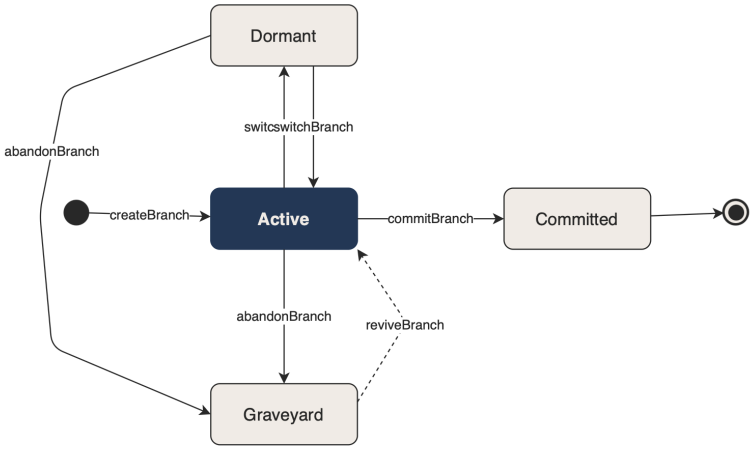
Use Case Diagram

US-1 is the foundational use case, included by US-2 and US-6. US-3 and US-5 extend US-1 and US-2 respectively. US-4 is standalone, as it consumes the graph but doesn't modify it.



Branch Lifecycle State Machine

A branch has four states: Active (currently being developed), Dormant (set aside but not committed or abandoned), Committed (chosen as canonical, merged into main story trunk), Graveyard (rejected but preserved for reference).



Transitions: createBranch → Active. switchBranch toggles between Active and Dormant. commitBranch moves Active to Committed (terminal). abandonBranch moves Active or Dormant to Graveyard. reviveBranch allows Graveyard → Active for writers returning to discarded ideas.

Non-Goals (explicitly out of scope for v1)

- Prose writing surface: Ampersand is a thinking tool, not a writing tool. v2 might explore an intelligent bridge from skeleton to prose.
- Voice drift detection: Research-grade problem, we would probably need PhDs.
- Mobile support: Desktop web only, for now.
- Real-time collaboration: Conflict resolution and operational transformation aren't worth solving for an individual-writer tool.
- Git-style version control: Branches are live exploration, not commits.
- Audio/voice input: Adds platform-specific complexity.
- Multi-document support: One story project per account in v1.

3. Quality Attribute Scenarios

QAS-1: Performance; Conversation responsiveness.

- Stimulus: The writer sends a conversation message containing one or more story decisions.
- Response measure: The system returns a natural-language response and reflects new graph nodes in the UI within 5 seconds at p95, under normal load and standard LLM API latency.

QAS-2: Reliability; Data persistence.

- Stimulus: The writer's session terminates mid-conversation (browser close, network drop, crash).
- Response measure: No more than the single in-flight conversation turn is lost. All previously-confirmed graph state is recoverable on return. Confirmation of persistence happens within 2 seconds of each successful turn.

QAS-3: Usability; Time-to-first-graph.

- Stimulus: A new writer signs up, opens a fresh project, and types their first message.
- Response measure: Within 90 seconds of signup, the writer sees their first populated graph node in the inspector, with no required tutorial step.

QAS-4: Security; User data isolation.

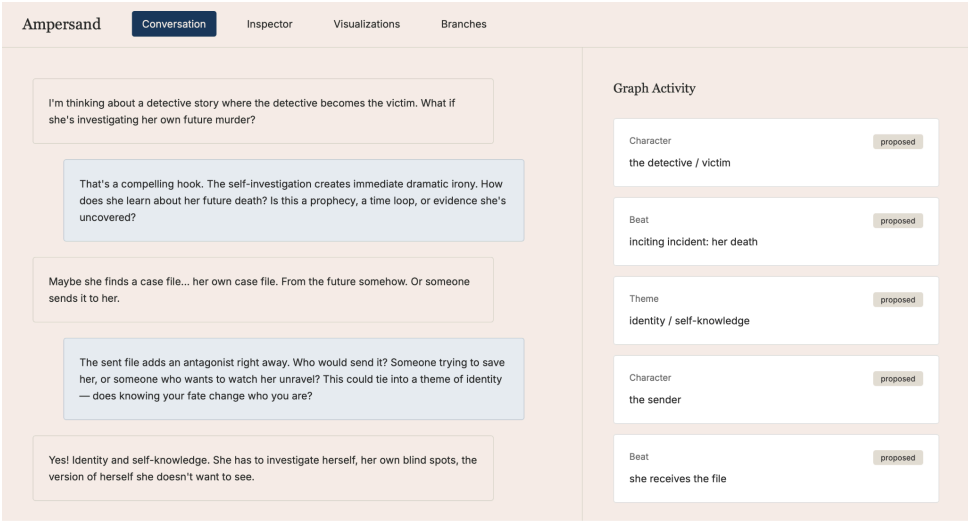
- Stimulus: An authenticated writer queries their story.
- Response measure: The system returns only data belonging to that account, including in AI-generated responses (LLM context scoped to the active user's graph). All persisted data is encrypted at rest; all client-server traffic uses TLS.

QAS-5: Functional Fidelity; End-to-end story development.

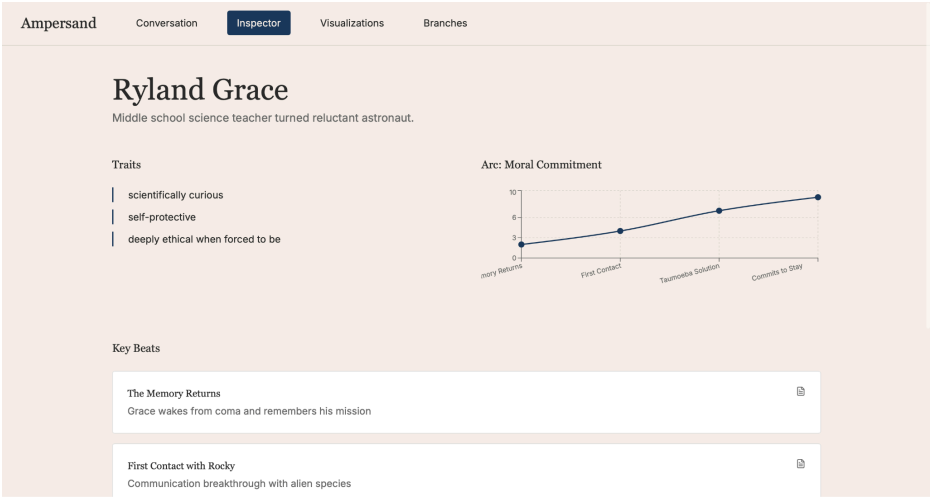
- Stimulus: A writer with no prior knowledge of the system uses Ampersand to develop a story from blank project to exported Markdown, working in sessions over multiple days, without external help.
- Response measure: The writer produces a Markdown export containing at least 3 characters, 5 ordered beats, 2 themes, and one branching decision they explored before committing, without manually correcting graph data the system extracted from their conversations. At least 90% of facts in the export are traceable to source conversation turns. Validated through user testing with at least 3 writers during implementation.

4. User Interaction Mockups

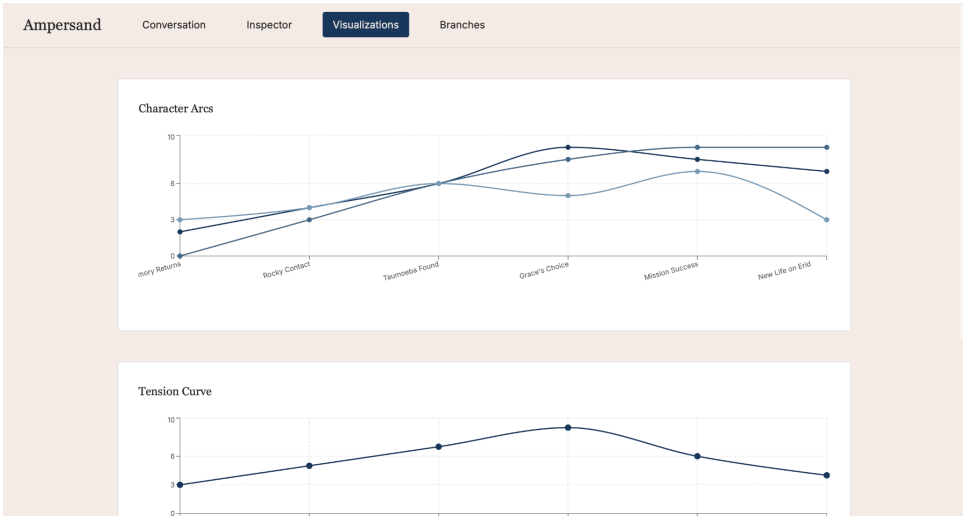
The four mockups below specify the primary surfaces, each rendered via Figma Make.



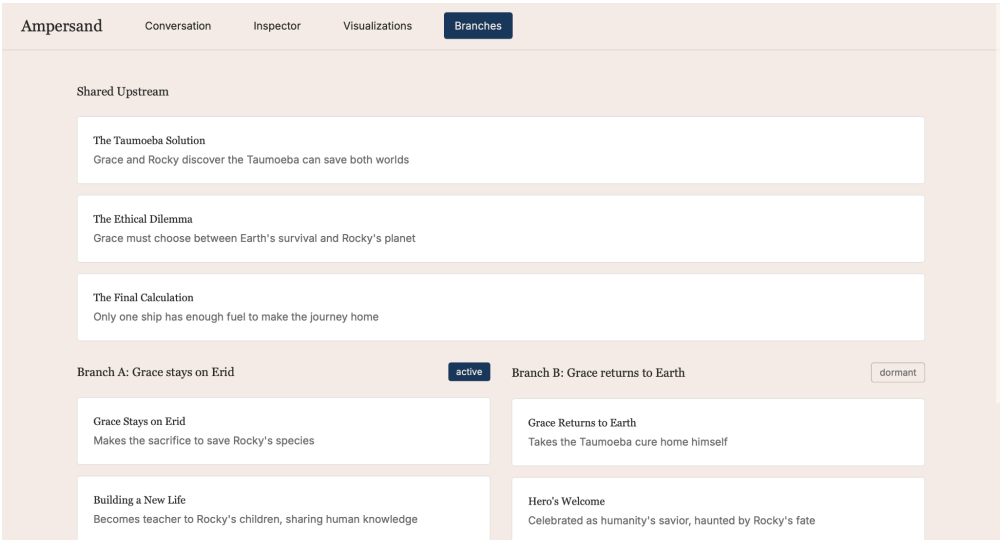
Mockup 1: Conversation interface (Figure 4). Two-panel layout: conversation thread on the left (~60% width), live "graph activity" panel on the right showing nodes as they're created.



Mockup 2: Character page (Figure 5). Name and one-line description at top; traits list upper-left; arc visualization upper-right; beat cards below, each with a citation marker.



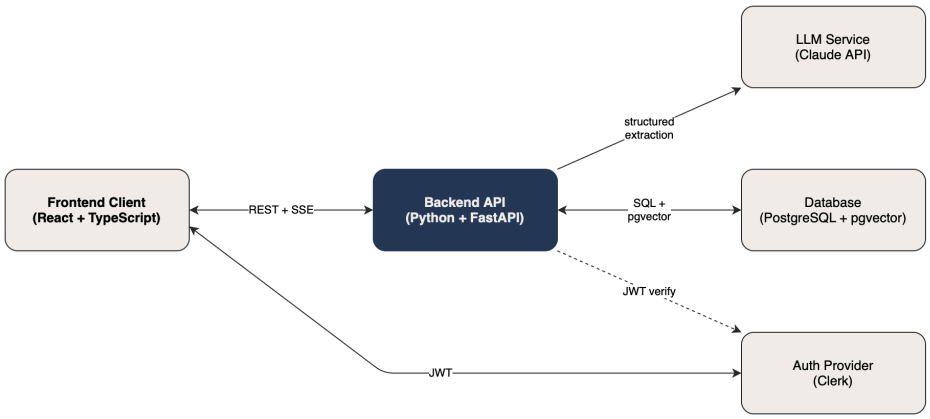
Mockup 3: Visualizations dashboard (Figure 6). Three stacked panels: multi-line character arcs graph, single-line tension curve, horizontal story-structure heatmap showing where development is dense versus sparse.



Mockup 4: Branch comparison view (Figure 7). Shared upstream context at the top; two parallel columns showing divergent beats per branch; commit-branch action bar at the bottom.

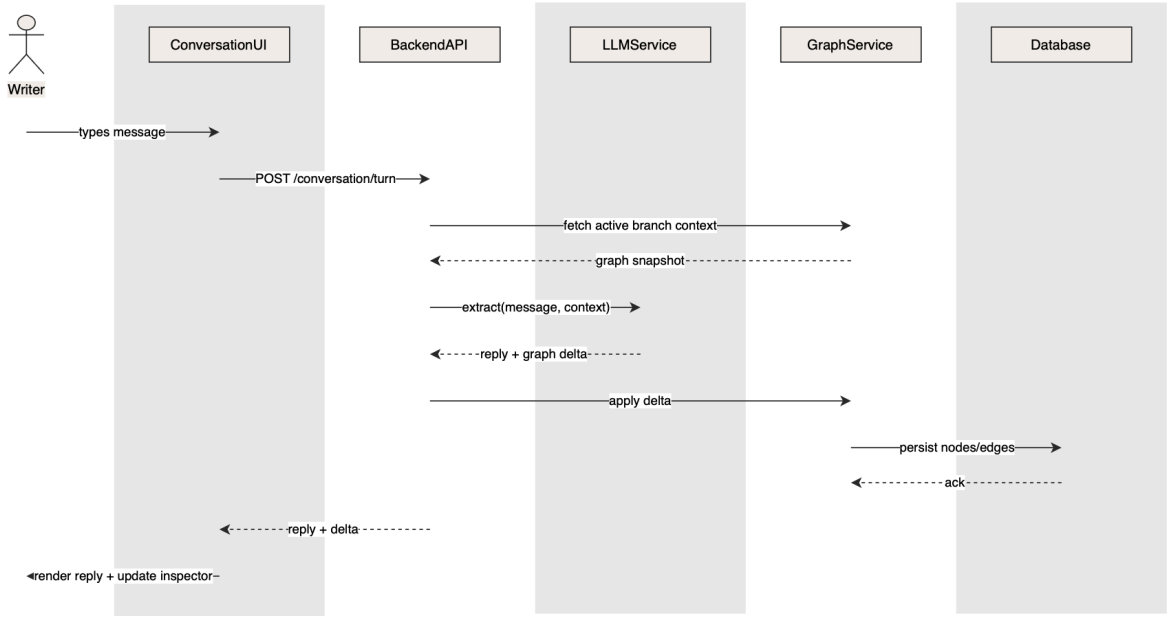
5. The Design

Ampersand is a desktop web app with three layers: a React/TypeScript frontend, a Python FastAPI backend, and a PostgreSQL persistence layer with pgvector. All graph queries go through the backend; the frontend never talks directly to the LLM. Real-time graph updates stream from server to client via SSE. See Figure 8 at the top of the next page for the component diagram.



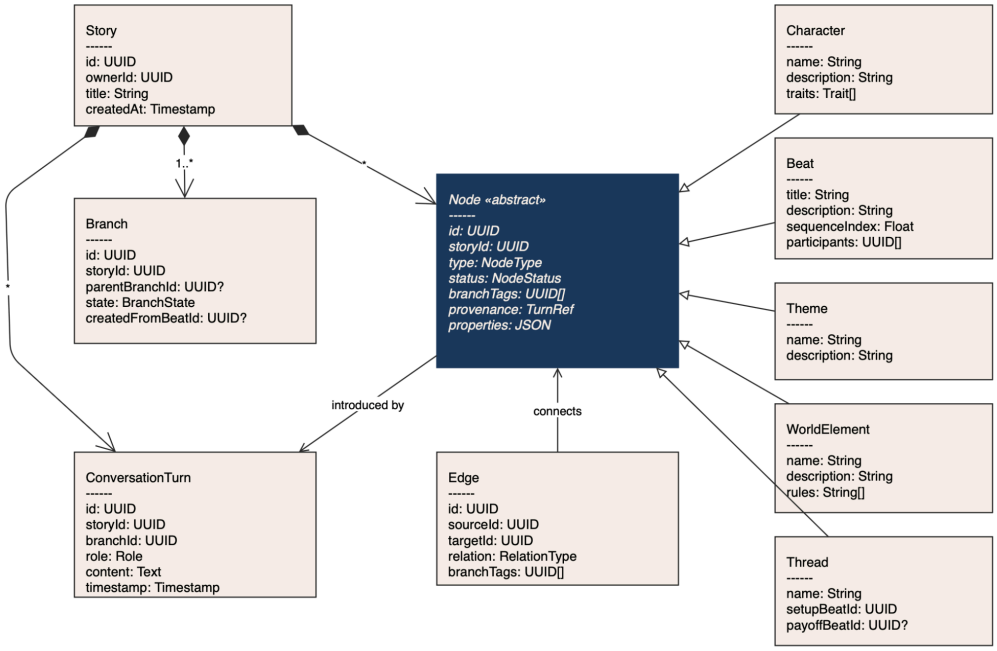
Sequence Diagram: Conversational Ingestion

The most interesting integration in the system is the round-trip from a writer's conversation message to a structured graph update. Figure 9, below, shows this flow.



The flow: the writer types a message; the UI sends it to the backend; the backend fetches the current branch context from the graph service; that context plus the message goes to the LLM service, which performs structured extraction returning both a natural-language reply and a graph delta; the backend applies the delta through the graph service, which persists changes and returns; the backend returns the reply and delta to the UI for rendering.

Class Diagram: Core Domain Model



Node is abstract. Character, Beat, Theme, WorldElement, and Thread are its concrete subtypes. Every node carries branchTags (the branches it belongs to), status (proposed/committed/rejected), and provenance (a reference to the conversation turn that introduced it). Edge represents typed relationships between nodes (e.g., character participates in beat). ConversationTurn is the origin of all extraction work.

Data Model

The story graph is stored in PostgreSQL. The schema reflects the class diagram with these tables:

- *Stories:* one row per story project, owned by a user.
- *Branches:* one row per exploration branch, with a state enum and an optional parent.
- *Nodes:* one row per graph node, with a type discriminator, a status enum, a branchTags array (UUID[]), a provenance reference, and a JSONB properties column holding type-specific fields.
- *Edges:* one row per relationship, with sourceId, targetId, relation enum, and branchTags.
- *Conversation_turns:* full history of writer/AI exchanges, scoped by branch.
- *Users:* managed by Clerk; the application stores only the Clerk user ID and a created-at timestamp.

The branchTags arrays on nodes and edges implement the per-node branch tagging strategy: a query for the active branch filters WHERE active_branch_id = ANY(branch_tags). This keeps branches lightweight (no graph forking) while supporting fast retrieval. pgvector columns on conversation_turns and nodes enable embedding-based retrieval for the conversational query interface (US-6).
This model is our current best design and the central design challenge of the project. Branch semantics in particular are subject to refinement during implementation; alternatives are discussed in the next section.

API

The backend exposes a RESTful API plus a single SSE endpoint for real-time graph updates. Authentication is via Clerk-issued JWTs; every request is scoped to the authenticated user's stories.

- *POST /conversation/turn*: Submit a writer message. Persists a ConversationTurn, applies extracted graph delta, broadcasts on SSE. Returns AI reply + delta. 404 if the story is not owned by the user; 429 on the rate limit.
- *GET /stories/{storyId}/graph*: Fetch graph state for the active branch (or specified branch).

- *POST /branches*: Create exploration branches. Returns a new branch in Active state.
- *POST /branches/{branchId}/transition*: Transition branch state per Figure 3. 400 on invalid transition.
- *GET /stories/{storyId}/export*: Export active branch as Markdown with citations. Optional includeAlternativeBranches.
- *POST /queries*: Natural-language question over the graph. Returns a grounded answer with citations or explicit "not in graph."
- *GET /stream/{storyId} (SSE)*: Real-time graph update stream.

Tech Stack

- **Frontend:** React + TypeScript + Next.js (team familiarity, Next.js for routing/SSR/Vercel deploy). Tailwind CSS with shadcn/ui for editorial design and accessible primitives. D3.js for non-standard visualizations.
- **Backend:** Python 3.11 + FastAPI. Strong LLM ecosystem (Anthropic SDK, instructor, Pydantic). Async support for SSE. The team is prepared to pivot to TypeScript/Node if velocity demands.
- **LLM:** Claude API, Haiku 4.5 default, Sonnet 4.6 for harder extraction passes.
- Database. PostgreSQL 16 + pgvector. Single store for relational graphs and embeddings; JSONB for flexible node properties.
- **Auth:** Clerk, managed, free tier.
- **Real-time SSE:** one-way streaming, native browser support, simpler than WebSockets for our needs.
- **Deployment:** Vercel (frontend), Railway (backend), Neon (Postgres). Free tiers cover the project.
- CI/CD: GitHub Actions with PR-based workflow and required code review on merge.

6. Alternatives Considered

We considered viable alternatives for four major design decisions.

Branch Implementation

Full graph forking (git-style) was the leading alternative, which would be duplicating the entire graph per branch. Simple to switch between, but storage scales with branch count and merging conflates narrative reconciliation with data merging. We chose per-node branch tags because branches are conceptually lightweight, writers create them casually and abandon most. Tagging keeps creation cheap; the read-path complexity hides behind query helpers.

Database

Neo4j was the obvious alternative. Cypher is more natural for graph traversals and graph databases handle deeply-connected queries efficiently. We chose PostgreSQL with pgvector because the team has Postgres experience (from 143) and little with Neo4j, our query patterns are mostly shallow, and we need vector retrieval for US-6; pgvector keeps that in the same store. If queries become traversal-heavy at scale, that's a v2 problem.

LLM

Fine-tuning a small open-weight model (Llama 3, Mistral) on a custom corpus would give us full control and avoid third-party dependence. We rejected it on timeline; corpus curation, fine-tuning, and validation is not something we think we can reliably get done in the timeframe. API endpoints should do the trick nicely anyways.

Frontend architecture

Server-rendered (HTMX, Hotwire) would simplify the client significantly and is faster to build. We chose React/TypeScript because Ampersand's interactions are not simple form submissions, it requires streaming responses, interactive D3 visualizations, live SSE updates while navigating between tabs. These are exactly where SPAs are the right tool.

7. Feasibility

API Familiarity

The team has working experience with React, TypeScript, Python, FastAPI, and PostgreSQL. New: the Claude API and pgvector. Both are well-documented with first-party SDKs. Week 1 of implementation is a foundation week in which the graph engine team validates Claude's structured extraction quality before downstream surfaces commit to the resulting API.

Performance

QAS-1's 5-second p95 is achievable: Claude Haiku 4.5 returns structured output in well under 2 seconds at our prompt sizes; Sonnet 4.6 (used for the hardest extraction passes) returns in 2–4 seconds; surrounding work is sub-100ms. Long conversations are mitigated by retrieving only the active branch's recent context per turn rather than full history.

Cost

LLM usage is the main driver. At Haiku 4.5 and Sonnet 4.6 pricing, an average 50-turn session costs $0.25–$0.50. Total spend across team and demo period: ~$25–150. Database, hosting, and auth all on free tiers.

Existing Projects

No tool combines structured story-graph extraction, branching exploration, and curatorial AI positioning. The closest tools fall into the three categories addressed in Context/Scope and none of them ingest conversation, build a story-aware graph, or surface gap analysis.

Third-party Reliability

Our LLM API is a critical dependency. If unavailable during the demo, we are probably screwed. Auth, database, and deployment are managed services with adequate free-tier reliability.

Useful Features

Three features at greatest risk of being built but unused: gap surfacing (US-5), branch comparison (US-3), and natural-language query (US-6). User testing during implementation per QAS-5 mitigates this, as features that don't engage testers get deprioritized for polish, with time reallocated to the core loop (conversation, character page, export).

Risk Summary

The largest unresolved risk is structured extraction quality, or the LLM's ability to reliably distinguish proposals from commitments, retractions from elaborations. This risk is concentrated in the graph engine, owned by the team's two strongest contributors, and is addressed in the foundation week before downstream surfaces commit to the API. Fallback if extraction quality is below threshold: surface confidence and status badges directly to the user and require explicit confirmation before nodes commit. This shifts a small UI burden to the writer in exchange for high-confidence graph state.